Wiki — Agendinha

Documentação técnica e funcional do sistema
de gerenciamento de consultas e exames

Backend:	`agendinha-backend-simple` (Django + DRF)
Frontend:	`agendinha-web` (Nuxt + Vue + TypeScript)
Guarda-chuva:	`agendinha` (submódulos)
Organização:	UnifespCodeLab

Documentação gerada a partir da análise do código-fonte real dos repositórios. Quando o código diverge de planejamentos anteriores, **o código é a fonte de verdade** e a divergência é sinalizada.

Versão 1.0 da documentação

Sumário

1 Visão Geral **2**
1.1 Composição da solução . . . 2
1.2 Origem do backend . . . 2
1.3 Fluxo geral . . . 2

2 Objetivos e Funcionalidades **2**
2.1 Objetivos . . . 2
2.2 Funcionalidades (conforme o código) . . . 3

3 Arquitetura do Sistema **3**
3.1 Visão de componentes . . . 3
3.2 Responsabilidades por camada . . . 4
3.3 Integrações externas . . . 4
3.4 Prefixo de rotas . . . 4

4 Tecnologias **4**
4.1 Backend . . . 4
4.2 Frontend . . . 5

5 Backend **5**
5.1 Estrutura de diretórios . . . 5
5.2 Principais arquivos . . . 6
5.3 Ciclo de uma requisição . . . 6

6 Referência da API **7**
6.1 Base URL e documentação . . . 7
6.2 Health Check . . . 7
6.3 Usuários e autenticação . . . 7
6.4 Responsáveis . . . 8
6.5 Agendamentos . . . 8
6.6 Notificações . . . 8
6.7 Web Push e páginas estáticas . . . 8
6.8 Exemplos . . . 9
6.9 Códigos de status . . . 9

7 Banco de Dados **9**
7.1 Diagrama entidade-relacionamento . . . 10
7.2 Entidades . . . 10
7.3 Migrations . . . 10

8 Autenticação e Segurança **11**
8.1 Geração do token . . . 11
8.2 Validação . . . 11
8.3 Autorização por papel . . . 11
8.4 Fluxo de login . . . 11
8.5 Senhas — BCrypt . . . 12
8.6 Login com Google OAuth2 . . . 12

9 Frontend **12**
9.1 Estrutura . . . 12
9.2 Páginas (rotas) . . . 12
9.3 Estado (Pinia) . . . 13
9.4 Integração com a API . . . 13
9.5 Calendário, notificações e push . . . 13

10 Variáveis de Ambiente **13**
10.1 Backend (`.env`) . . . 13
10.2 Frontend (`.env`) . . . 14

11 Instalação e Execução **14**
11.1 Clonar com submódulos . . . 14
11.2 Backend — manual . . . 14
11.3 Backend — Docker Compose . . . 15
11.4 Frontend . . . 15

12 Testes **15**
12.1 Backend — pytest . . . 15
12.2 Frontend — Vitest . . . 15
12.3 Teste de integração (fluxo completo) . . . 16

13 Notificações **16**
13.1 Geração automática . . . 16
13.2 Ciclo de vida . . . 16
13.3 Web Push e cron . . . 16

14 Desenvolvimento e Manutenção **17**
14.1 Fluxo de contribuição . . . 17
14.2 Submódulos . . . 17
14.3 Dívida técnica (encontrada no código) . . . 17
14.4 Regra de manutenção da documentação . . . 17

15 Fluxogramas complementares **17**
15.1 Cadastro e confirmação de conta . . . 17
15.2 Login com Google OAuth2 . . . 18
15.3 Proteção de rota no frontend . . . 18

16 Licenciamento de Software **18**
16.1 Licença dos próprios projetos . . . 19
16.2 Backend — dependências Python . . . 19
16.3 Frontend — dependências JavaScript/TypeScript . . . 19
16.3.1 Dependências de runtime . . . 19
16.3.2 Dependências de desenvolvimento . . . 20
16.4 Pontos de atenção e obrigações . . . 20
16.5 Arquivos e atualização do inventário . . . 21

17 Glossário e Próximos Passos **21**
17.1 Glossário . . . 21
17.2 Próximos passos sugeridos . . . 22
17.3 Regra de manutenção . . . 22

1 Visão Geral

O **Agendinha** é uma aplicação para auxiliar pacientes (e seus responsáveis) no gerenciamento de **consultas médicas e exames**. O sistema centraliza esses compromissos em uma agenda digital, permitindo consultar, cadastrar e gerenciar agendamentos, visualizá-los em um calendário e receber **notificações push** de lembrete.

O projeto é desenvolvido pela **UnifespCodeLab**. A aplicação é identificada na interface como "Agendinha".

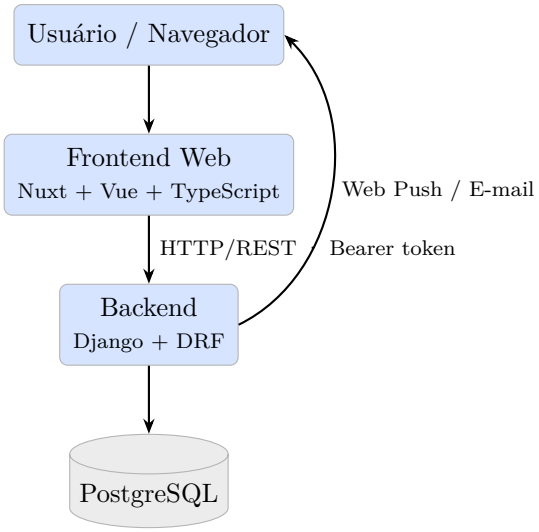
1.1 Composição da solução

- `agendinha-web` — frontend web (Nuxt + Vue + TypeScript). Interface de interação; consome a API via HTTP/REST.
- `agendinha-backend-simple` — backend (Python + Django + Django REST Framework). Expõe a API, aplica regras de negócio, autentica/ autoriza, persiste dados e dispara notificações.
- `agendinha` — repositório guarda-chuva que referencia os dois acima como **submódulos git** (`front` → `agendinha-web`, `back` → `agendinha-backend-simple`).

1.2 Origem do backend

O backend atual é uma **API Django unificada** que substituiu uma arquitetura anterior baseada em **4 microserviços Java Spring Boot** (orquestrador, usuários, agendamentos e notificações). Diversos comentários no código referenciam a migração ("Migrado de: ... Controller"), e os tokens JWT foram mantidos **compatíveis** com os emitidos pelos serviços Java.

1.3 Fluxo geral



Nota. O documento de planejamento inicial falava em "Paciente" como entidade. No código real, a entidade associada ao usuário chama-se `Responsavel` (responsável) e é exposta sob o prefixo `/responsaveis`. O "paciente" é, na prática, o próprio `Usuario`. Esta Wiki usa a terminologia do código.

2 Objetivos e Funcionalidades

2.1 Objetivos

- Centralizar consultas e exames em uma única agenda digital.

- Facilitar a visualização dos compromissos (listas por semana/futuros e calendário).
- Permitir o cadastro e o gerenciamento de agendamentos (feito por administradores).
- Lembrar o usuário de compromissos próximos por notificações.
- Proteger os dados por autenticação (JWT) e autorização por papéis.
- Oferecer interface web organizada e responsiva (PWA).

2.2 Funcionalidades (conforme o código)

Contas e autenticação: pré-cadastro por administrador (CPF + nome); conclusão de cadastro pelo usuário (e-mail/senha) com e-mail de confirmação; confirmação via link (`/conta/:id/:token`); login por e-mail/senha; login com **Google OAuth2**; redefinição de senha (pedido por e-mail e troca via `/senha/:id/:token`); troca de senha autenticado; atualização de perfil e foto; exclusão da conta; cadastro e login de **administradores**.

Responsáveis (`Responsavel`): criar, listar, atualizar e excluir (ADMIN); buscar por nome ou ID (público).

Agendamentos: criar, consultar, atualizar e excluir (ADMIN); listar todos (ADMIN); listar os do próprio usuário (USER); exportar para o **Google Calendar**; visualização em calendário (frontend).

Notificações: criação automática de 4 lembretes por agendamento (1 semana, 3 dias, 1 dia e 4 horas antes — apenas datas futuras); criação em lote; listar por agendamento e por usuário; listar não lidas; marcar como lida; excluir; **Web Push** (service worker + VAPID).

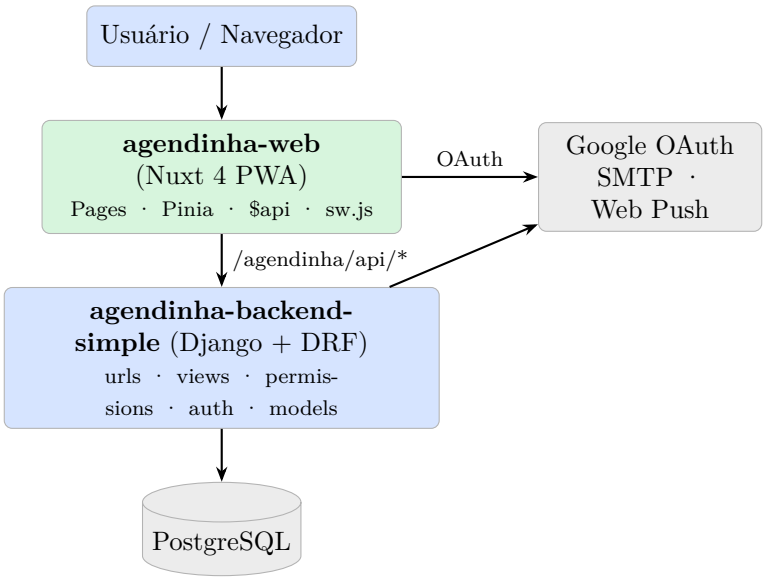
Controle de acesso: papéis `ROLE_USER` e `ROLE_ADMIN`; proteção de rotas no backend *e* no frontend.

[!] **Atenção.** O PDF inicial listava endpoints como `/usuarios/pacienteid/registrar` e `/notificacoes/{id}/lida`. O código real usa `/responsaveis`, pré-cadastro por admin e caminhos ligeiramente diferentes. Ver a Seção 6.

3 Arquitetura do Sistema

A arquitetura é **cliente-servidor**. O frontend faz chamadas HTTP/REST para a API (Django/-DRF), que aplica regras de negócio, valida, autentica/autoriza e acessa o PostgreSQL. Serviços externos (Google, SMTP, Web Push) são integrados pelo backend e/ou pelo frontend.

3.1 Visão de componentes



3.2 Responsabilidades por camada

Camada	Responsabilidade
Frontend	Renderizar páginas/componentes, capturar entradas, validar na interface, gerenciar estado/sessão (token em cookie), exibir calendário e notificações, registrar o service worker de push, iniciar o OAuth do Google.
Backend	Expor a API REST, validar dados (serializers), aplicar regras de negócio, autenticar via JWT e autorizar por papel, gerar notificações, enviar e-mails e push, integrar com o Google.
Banco de dados	Persistir `Usuario`, `Responsavel`, `Agendamento`, `Notificacao` e `PushSubscription`.

3.3 Integrações externas

Google OAuth2 / Google Calendar; SMTP (Gmail) para e-mails de confirmação e redefinição; Web Push (VAPID); e um serviço **cron** (Docker) para envio de lembretes (`manage.py send_reminders`).

3.4 Prefixo de rotas

Todas as rotas ficam sob `/agendinha/api/` (em `core/urls.py`). O Swagger está em `/api/docs/`, o Redoc em `/api/redoc/` e o schema em `/api/schema/`.

4 Tecnologias

As versões vêm de `requirements.txt` (backend) e `package.json` (frontend).

4.1 Backend

Área	Tecnologia	Versão
Linguagem	Python	3.12 (Docker)
Framework web	Django	5.2.8
API REST	Django REST Framework	3.15.2
JWT (SimpleJWT)	djangorestframework-simplejwt	5.3.1
JWT customizado	PyJWT	2.10.1
Banco / driver	PostgreSQL / psycopg2-binary	2.9.10
Doc API	drf-spectacular	0.27.2
CORS	django-cors-headers	4.5.0
Senhas	bcrypt	4.2.1
Env vars	python-dotenv	1.2.1
Testes	pytest-django	4.9.0
Imagem	pillow	12.1.0
OAuth/Google	django-allauth / google-auth / google-api-python-client	65.14.3 / 2.48.0 / 2.191.0
Web Push	pywebpush	2.3.0

4.2 Frontend

Área	Tecnologia	Versão
Framework	Nuxt	^4.5.2
UI	Vue 3 + Vuetify	vuetify ^3.7.0
Estado	Pinia + @pinia/nuxt	2.2 / ^0.7.0
Build	Vite + vite-plugin-vuetify	^2.1.1
Calendário	v-calendar	^3.1.2
Login Google	vue3-google-login / google-auth-library	^2.0.33
PWA / Push	@vite-pwa/nuxt	^1.1.1
HTTP	ofetch ($fetch nativo)	—
UX	vue-toast-notification / moment	^3.1.3 / ^2.30.1
Tipagem	TypeScript	^5.8.3
Testes	Vitest / @vue/test-utils	^3.2.4 / ^2.4.6

[!] **Atenção.** Dependências instaladas mas não usadas no código: `@schedule-x/*` (o calendário efetivo é o `v-calendar`) e `jest`/`ts-jest` (o runner é o Vitest). Candidatas à remoção.

5 Backend

O backend é uma API em Python com **Django** e **Django REST Framework**. O projeto Django chama-se `core` e o app principal, `restfull`.

5.1 Estrutura de diretórios

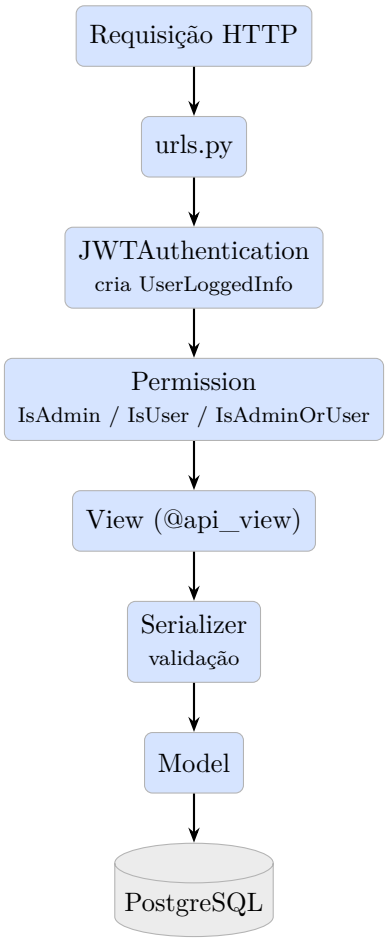
```
agendinha-backend-simple/
|- Dockerfile docker-compose.yml requirements.txt .env.example
`- agendinha/
   |- manage.py pytest.ini
   |- core/ # settings.py, urls.py, wsgi.py, asgi.py
   |- restfull/
   | |- models.py # entidades persistidas
   | |- authentication.py # JWT customizado
   | |- permissions.py # IsAdmin / IsUser / IsAdminOrUser / IsOwnerOrAdmin
   | |- urls.py # rotas da API
   | |- admin.py # Django Admin
   | |- services.py utils.py schema.py conftest.py test_*.py
   | |- serializers/ # user admin guardian appointment notification
   | |- views/ # user admin guardian appointment
   | | # notification subscription utils
   | `- migrations/
   |- templates/ # e-mails HTML e paginas estaticas
   `- static/
```

Nota. Os `serializers` e as `views` — monolíticos no planejamento — estão de fato **modularizados por domínio** em pacotes.

5.2 Principais arquivos

Arquivo	Responsabilidade
`models.py`	Entidades (`Usuario`, `Responsavel`, `Agendamento`, `Notificacao`, `PushSubscription`) e lógica de senha/-token.
`serializers/`	Validação e transformação de entrada/saída por domínio.
`views/`	Lógica dos endpoints (function-based views com `@api_view`).
`urls.py`	Rotas da API.
`authentication.py`	`JWTAuthentication` e `UserLoggedInfo`.
`permissions.py`	Autorização por papel.
`admin.py`	Registro no Django Admin.
`core/settings.py`	Configuração central.

5.3 Ciclo de uma requisição



A autenticação não busca o usuário no banco a cada requisição: monta um objeto leve `UserLoggedInfo` a partir dos claims do token. Views que precisam do registro completo recarregam-no com `Usuario.objects.get(...)`.

[!] **Atenção. Bug latente:** em `notification_create`, a notificação é criada com `user=user`, mas o campo do model é `usuario`. Além disso, em `settings.py` a `SECRET_KEY` é reatribuída para a string fixa `"agendinha"` após ler o ambiente. Ver Seções 8 e 14.

6 Referência da API

Derivada de `restfull/urls.py` e das views. A fonte operacional definitiva é a documentação **Swagger/OpenAPI**.

6.1 Base URL e documentação

```
Base da API: /agendinha/api/
Swagger UI: /api/docs/
Redoc: /api/redoc/
Schema JSON: /api/schema/
# Dev (padrao): http://localhost:8000/agendinha/api/
```

Endpoints protegidos exigem `Authorization: Bearer <token>`. A coluna **Auth** indica o papel: × público, USER, ADMIN ou U/A (qualquer autenticado).

6.2 Health Check

Método	Endpoint	Descrição	Auth
GET	`/hello`	Teste de conectividade	×

6.3 Usuários e autenticação

Método	Endpoint	Descrição	Auth
POST	`/usuarios/registrar`	Conclui pré-cadastro; dispara e-mail	×
POST	`/usuarios/login`	Login e-mail/senha	×
POST	`/usuarios/login/google`	Login via Google OAuth2	×
POST	`/usuarios/confirmar`	Confirma cadastro (por id ou e-mail)	×
POST	`/usuarios/foto`	Atualiza foto (multipart)	U/A
POST	`/usuarios/email/redefinir-senha`	Solicita e-mail de redefinição	×
PUT	`/usuarios/redefinir-senha`	Redefine a senha	×
GET	`/usuarios`	Dados do usuário + notificações	U/A
GET	`/usuarios/pesquisar?nome=`	Busca pacientes por nome	ADMIN
PUT	`/usuarios/update`	Atualiza nome/e-mail/responsável	U/A
DELETE	`/usuarios/delete`	Exclui o usuário autenticado	U/A

Admin

Método	Endpoint	Descrição	Auth
POST	`/admin/registrar`	Registra um administrador	×
POST	`/admin/login`	Login de administrador	×
POST	`/admin/usuarios/registrar`	Pré-cadastra usuário (CPF + nome)	ADMIN

6.4 Responsáveis

Método	Endpoint	Descrição	Auth
GET	`/responsaveis`	Lista responsáveis	ADMIN
POST	`/responsaveis/create`	Cria responsável	ADMIN
PUT	`/responsaveis/<id>`	Atualiza responsável	ADMIN
DELETE	`/responsaveis/<id>/delete`	Exclui responsável	ADMIN
POST	`/responsaveis/pesquisar`	Busca por nome	×
GET	`/responsaveis/pesquisar/<id>`	Busca por ID	×

6.5 Agendamentos

Método	Endpoint	Descrição	Auth
POST	`/agendamentos/create`	Cria agendamento	ADMIN
GET	`/agendamentos`	Lista todos	ADMIN
GET	`/agendamentos/<id>`	Consulta um agendamento	U/A
PUT	`/agendamentos/<id>/update`	Atualiza	ADMIN
DELETE	`/agendamentos/<id>/delete`	Exclui	ADMIN
GET	`/agendamentos/usuario`	Lista os do usuário	USER
POST	`/agendamentos/google`	Exporta ao Google Calendar	U/A

6.6 Notificações

Método	Endpoint	Descrição	Auth
POST	`/notificacoes`	Cria notificações automáticas	ADMIN
POST	`/notificacoes/conjunto`	Cria em lote	ADMIN
GET	`/notificacoes/<id\_agendamento>`	Lista por agendamento	U/A
GET	`/notificacoes/usuario/<id\_usuario>`	Lista por usuário	U/A
GET	`/notificacoes/id/<id\_notificacao>`	Exclui por ID (via GET)	U/A
POST	`/notificacoes/naoLidas`	Lista não lidas (futuras)	U/A
POST	`/notificacoes/<id>/lida`	Marca como lida	U/A
DELETE	`/notificacoes/<id\_ag>/delete`	Exclui todas de um agendamento	ADMIN

6.7 Web Push e páginas estáticas

Método	Endpoint	Descrição	Auth
POST	`/salvar-inscricao`	Salva inscrição push	U/A
GET	`/criar-responsavel`	Form HTML (template Django)	—
GET	`/criar-agendamento`	Form HTML (template Django)	—

Método	Endpoint	Descrição	Auth
GET	`/criar-notificacao`	Form HTML (template Django)	—

6.8 Exemplos

```
# Login de usuario
curl -X POST http://localhost:8000/agendinha/api/usuarios/login \
  -H "Content-Type: application/json" \
  -d '{"email": "usuario@email.com", "senha": "senha123"}'
```

Resposta 200:

```
{ "usuario": { "id_usuario": 1, "role": "ROLE_USER", "...": "..." },
  "notificacoes": [ ],
  "token": "eyJhbGciOi..." }
```

```
# Criar agendamento (ADMIN) - data no formato dd/MM/yyyy HH:mm
curl -X POST http://localhost:8000/agendinha/api/agendamentos/create \
  -H "Authorization: Bearer SEU_TOKEN" -H "Content-Type: application/json" \
  -d '{"titulo":"Consulta","descricao":"Retorno","data":"25/12/2026 14:30",
       "local":"Sala 3","medico":"Dra. Ana","id_usuario":1}'
```

6.9 Códigos de status

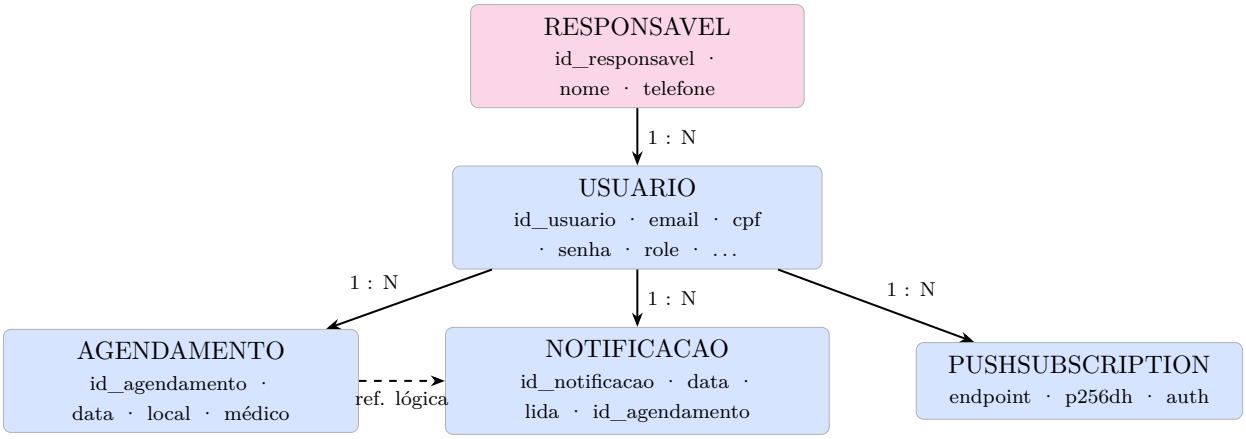
Código	Significado no sistema
200	Sucesso (algumas criações retornam 200, não 201)
204	Sem conteúdo (lista vazia / recurso inexistente em alguns GETs)
400	Erro de validação / credenciais inválidas
401	Não autorizado (conta Google em login por senha; senha atual incorreta)
403	Proibido (cadastro já confirmado; e-mail já existe sem Google)
500	Erro interno (recurso não encontrado; falha de conversão de data)

Nota. O backend usa `500` em casos que semanticamente seriam `404`. Ver sugestões na Seção 17.

7 Banco de Dados

Usa **PostgreSQL**. O modelo reflete `restfull/models.py`.

7.1 Diagrama entidade-relacionamento



Nota. `Notificacao.id_agendamento` é um `BigIntegerField` simples, **não** uma `ForeignKey`. A associação é lógica (por valor).

7.2 Entidades

`usuario` — identificação e autenticação:

Campo	Tipo	Observações
id_usuario	BigAutoField (PK)	
nome	CharField(255)	blank
email	EmailField	indexado
cpf	CharField(14)	**unique**, blank
senha	CharField(255)	hash **BCrypt**
cadastro_confirmado	Boolean	default False
responsavel	FK → Responsavel	SET_NULL
role	CharField	ROLE_USER (default) / ROLE_ADMIN
foto_perfil	ImageField	upload em images/
modo_google	Boolean	conta via Google
timeout_seconds	BigInteger	validade do token de e-mail
ativar_notificacoes_consultas	Boolean	default True

Métodos: `set_password`/`check_password` (BCrypt); `make_token`/`check_token` (HMAC-SHA256 para links de e-mail); `save_image_from_url` (foto do Google).

`responsavel`: `id_responsavel` (PK), `nome`, `telefone` (unique).

`agendamento`: `id_agendamento` (PK), `titulo`, `descricao`, `data` (indexado, ordenação padrão), `local`, `medico` (nullable), `usuario` (FK, SET_NULL), `lembrete_enviado` (usado pelo job de lembretes).

`notificacao`: `id_notificacao` (PK), `id_agendamento` (indexado), `data`, `lida`, `titulo`, `descricao`, `usuario` (FK, SET_NULL).

`pushsubscription`: `id` (PK), `usuario` (FK), `endpoint`, `p256dh`, `auth`.

7.3 Migrations

`0001_initial` (criação), `0002_alter_usuario_cpf`, `0003_alter_usuario_email`.

[!] **Atenção.** Divergência: o PDF modelava `PACIENTE` entre `USUARIO` e `AGENDAMENTO`. No código, o `Agendamento` referencia diretamente o `Usuario`; a entidade adicional é o `Responsavel`.

8 Autenticação e Segurança

A API usa **JWT**. Após o login, o cliente recebe um token enviado em `Authorization: Bearer <token>`.

8.1 Geração do token

Assinado com **HS256** e a chave `SECURITY_TOKEN`. Claims:

```
{ "sub": "usuario@email.com", "iss": "agendinha-api-django",
  "idUsuario": 123, "role": "ROLE_USER",
  "iat": 1700000000, "exp": 1700086400 } // exp = agora + 24h
```

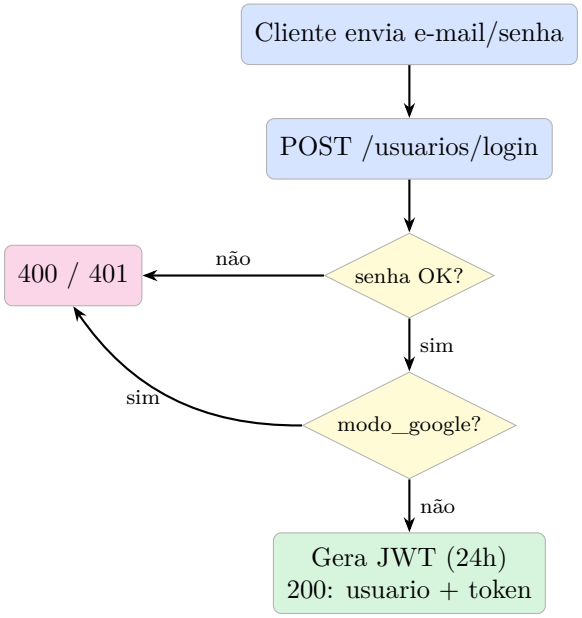
8.2 Validação

`JWTAuthentication` lê o header, decodifica com `SECURITY_TOKEN` + HS256 validando o emissor (com *fallback* sem validar `iss` para compatibilidade com tokens Java), exige `sub` e `idUsuario`, e monta `UserLoggedInfo` como `request.user` — sem ida ao banco.

8.3 Autorização por papel

Classe	Regra
IsAdmin	role == ROLE_ADMIN
IsUser	role == ROLE_USER
IsAdminOrUser	role ∈ {ROLE_ADMIN, ROLE_USER}
IsOwnerOrAdmin	admin, ou dono (`obj.id_usuario == request.user.id_usuario`)

8.4 Fluxo de login



8.5 Senhas — BCrypt

Senhas são armazenadas com BCrypt (`bcrypt.hashpw/checkpw`), compatível com o `BCryptPasswordEncoder` do Java. Nunca em texto puro.

8.6 Login com Google OAuth2

Frontend: inicia o *code flow*, troca o código por tokens e chama `POST /usuarios/login/google` com `{token(id_token), email}`. *Backend*: valida o `id_token`, faz `get_or_create` do usuário (`modo_google=True` em contas novas, baixa a foto) e emite o JWT. Contas sem `modo_google` recebem `403`.

[!] **Atenção. Alertas de segurança no código:** (1) `SECRET_KEY = "agendinha"` fixa vira *fallback* de `SECURITY_TOKEN`; defina `SECURITY_TOKEN` por ambiente. (2) `.env` versionado no backend. (3) `googleClientSecret` e `vapidPrivateKey` expostos em `runtimeConfig.public` do Nuxt (enviados ao cliente). (4) `CORS_ALLOW_ALL_ORIGINS = DEBUG` — garanta `DEBUG=False` em produção.

9 Frontend

Aplicação **Nuxt 4** (PWA) com **Vue 3**, **TypeScript** e **Vuetify**. Estado em Pinia; consome a API via `$fetch`.

9.1 Estrutura

```
agendinha-web/
|- nuxt.config.ts package.json vitest.config.ts router.options.ts
|- .env.example Makefile
|- public/ (sw.js, images/) shared/types/ (interfaces TS)
`- app/
   |- app.vue # loader global, header condicional, transicoes
   |- pages/ # rotas por arquivo
   |- components/ # AppointmentCard(Generator), NotificationCard/Generator,
   | # TheHeader, ProfileImage, DeleteAccount
   |- composables/ # useApi, useAppointments, useNotification, useUser,
   | # useGoogleCalendar, usePush
   |- stores/ # auth.ts, loader.ts (Pinia)
   |- middleware/ # auth.ts (guarda de rota + papel)
   |- plugins/ # api.ts, vcalendar.client.ts, vuetify.ts, toast.ts
   `- utils/ # date, convertToISODate, urlBase64ToUint8Array, ...
```

9.2 Páginas (rotas)

Rota	Função
/boas-vindas	Onboarding (sem token e `firsttime` diferente de `false`)
/login	Login e-mail/senha e Google
/registro	Conclusão de cadastro
/completar-informacoes	Dados faltantes após cadastro Google
/	Home do paciente: abas Agendamentos e Calendário (ROLE_USER)
/admin	Painel admin: criar/editar agendamentos + calendário (ROLE_ADMIN)

Rota	Função
/contatos	Lista de contatos
/notificacoes	Lista de notificações
/configurar-notificacoes	Preferências (cookies)
/perfil, /editar-perfil	Ver/editar perfil, foto, excluir conta
/alterar-senha	Troca de senha autenticado
/pedido-alteracao-senha	Solicita redefinição por e-mail
/conta/:id/:token	Confirmação de conta (link do e-mail)
/senha/:id/:token	Redefinição de senha (link do e-mail)

9.3 Estado (Pinia)

auth.ts: `user`, `authenticated`, `notifications`, `notReadNotifications`, `googleTokens`. Ações: `refreshAuth` (GET /usuarios), `authenticateUser` (POST /usuarios/login, grava cookie `token`), `authenticateUserGoogle`, `logout`. **loader.ts**: `isLoading` + controle do overlay global.

9.4 Integração com a API

Três camadas sobre o `$fetch`: (1) `runtimeConfig.public.apiBase` (`NUXT_PUBLIC_API_BASE`, fallback `http://localhost:8080/`); (2) `plugins/api.ts` — `$api` com retorno normalizado `{status,data,...}` (3) `composables/useApi.ts` — injeta `Authorization: Bearer <token>` do cookie.

[!] **Atenção.** O backend serve a API sob `/agendinha/api/`, mas o fallback do frontend é a raiz `http://localhost:8080/`. Configure `NUXT_PUBLIC_API_BASE` para o host **e prefixo** corretos (ex.: `http://localhost:8000/agendinha/api/`).

9.5 Calendário, notificações e push

Usa **v-calendar v3** (client-only); agendamentos viram `attributes` (barra rosa `#E32585`). Export para o Google via `useGoogleCalendar`. PWA via `@vite-pwa/nuxt`; `usePush` registra `/sw.js`, assina o push com a chave VAPID e envia a inscrição a `POST /salvar-inscricao`; `sw.js` trata `push` e `notificationclick`.

10 Variáveis de Ambiente

Mantenha configurações sensíveis em variáveis de ambiente. **Valores reais não devem ser publicados** na Wiki, no Git ou em capturas de tela.

10.1 Backend (`.env`)

Variável	Finalidade	Default
DEBUG	Modo debug	True
`DJANGO_SECRET_KEY`	Chave do Django	insegura embutida
`SECURITY_TOKEN`	Assinatura do JWT (HS256)	cai no SECRET_KEY
`SECURITY_EMISSOR`	Emissor (iss) esperado	agendinha-api-django
`DATABASE_NAME/USER/PASSWORD`	Banco	agendinha_db / ...

Variável	Finalidade	Default
DATABASE_HOST/PORT	Host/porta	localhost / 5432
CORS_ALLOWED_ORIGINS	Origens (CSV)	localhost:3000,8080
ALLOWED_HOSTS	Hosts (CSV)	localhost,127.0.0.1
FRONTEND_URL	Base do front (links e-mail)	localhost:3000
SERVER_PORT	Porta (docker-compose)	8000
GOOGLE_CLIENT_ID/SECRET	OAuth2	—
GOOGLE_TOKEN_URI	Token URI do Google	—
EMAIL_HOST_USER/PASSWORD	SMTP (Gmail)	—
VAPID_PUBLIC/PRIVATE_KEY	Web Push	—
VAPID_ADMIN_EMAIL	Contato das claims VAPID	—
REMINDER_CRON_SCHEDULE	Agenda do cron (Docker)	—

SMTP fixado em `smtp.gmail.com:587` (TLS) no `settings.py`.

10.2 Frontend (`.env`)

Variável	Finalidade
NUXT_PUBLIC_API_BASE	URL base da API (fallback localhost:8080/)
NUXT_GOOGLE_CLIENT_ID	Client ID do OAuth
NUXT_GOOGLE_CLIENT_SECRET	Client Secret (△ ver aviso)
NUXT_VAPID_PUBLIC_KEY	Chave pública de push
NUXT_VAPID_PRIVATE_KEY	Chave privada de push (△ ver aviso)

[!] **Atenção.** `NUXT_GOOGLE_CLIENT_SECRET` e `NUXT_VAPID_PRIVATE_KEY` estão sob `runtimeConfig.public` — embarcados no bundle do cliente. Segredos não deveriam ser públicos.

11 Instalação e Execução

Pré-requisitos: backend — Python 3.10+ (Docker usa 3.12), PostgreSQL 14+, Git; frontend — Node.js + npm.

11.1 Clonar com submódulos

```
git clone --recursive https://github.com/UnifespCodeLab/agendinha.git
# ou, apos clonar sem --recursive:
git submodule update --init --recursive
```

11.2 Backend — manual

```
git clone https://github.com/UnifespCodeLab/agendinha-backend-simple.git
cd agendinha-backend-simple
python -m venv env && source env/bin/activate # Windows: env\Scripts\activate
pip install -r requirements.txt
cp .env.example .env # preencha os valores
# crie o banco PostgreSQL (DATABASE_NAME)
cd agendinha
python manage.py makemigrations && python manage.py migrate
python manage.py createsuperuser # opcional
python manage.py runserver # http://localhost:8000
```

API em `/agendinha/api/`; Swagger em `/api/docs/`; Admin em `/admin/`. O `manage.py` está em `agendinha-backend-simple/agendinha/`.

11.3 Backend — Docker Compose

Sobe **postgres**, **api** e **cron** (lembretes).

```
cp .env.example .env
docker-compose up -d
docker-compose down
```

[!] **Atenção.** O `Dockerfile`/`docker-compose.yml` ainda referenciam diretórios de nomes antigos, enquanto o projeto usa `agendinha/`. Ajuste `working_dir` e o caminho do cron antes de usar o build Docker.

11.4 Frontend

```
git clone https://github.com/UnifespCodeLab/agendinha-web.git
cd agendinha-web
npm install # postinstall roda "nuxt prepare"
cp .env.example .env # configure NUXT_PUBLIC_API_BASE
npm run dev # http://localhost:3000 (dev:host p/ rede local)
```

Scripts: `dev`, `dev:host`, `build`, `generate`, `preview`, `lint:eslint`, `lint:prettier`, `lint:fix`, `lint:full`.

12 Testes

12.1 Backend — pytest

Config em `pytest.ini` (`DJANGO_SETTINGS_MODULE = core.settings`, `testpaths = restfull`). Arquivos e nº de funções `test_*`:

Arquivo	Testes
test_auth.py	18
test_appointments.py	19
test_notifications.py	17
test_patients.py	17

```
cd agendinha-backend-simple/agendinha && source ../env/bin/activate
pytest # todos
pytest restfull/test_auth.py # um arquivo
pytest --cov=restfull --cov-report=html
```

[!] **Atenção. Testes desatualizados.** `conftest.py`/`test_*.py` importam `Paciente` e usam `id_paciente`/`paciente=` — entidades inexistentes hoje (hoje é `Responsavel`, e o agendamento liga-se a `Usuario`). A suíte tende a falhar na coleta. Migrar para o modelo atual.

12.2 Frontend — Vitest

`vitest.config.ts` define projetos `unit` (`test/{e2e,unit}/*`) e `nuxt` (`test/nuxt/*`). Ainda **não existe** o diretório `test/` — não há testes implementados; rode com `npx vitest`.

12.3 Teste de integração (fluxo completo)

`cadastro` → `login` → `criação de agendamento (admin)` → `geração de notificação` → `push/consult` → `marcação como lida`.

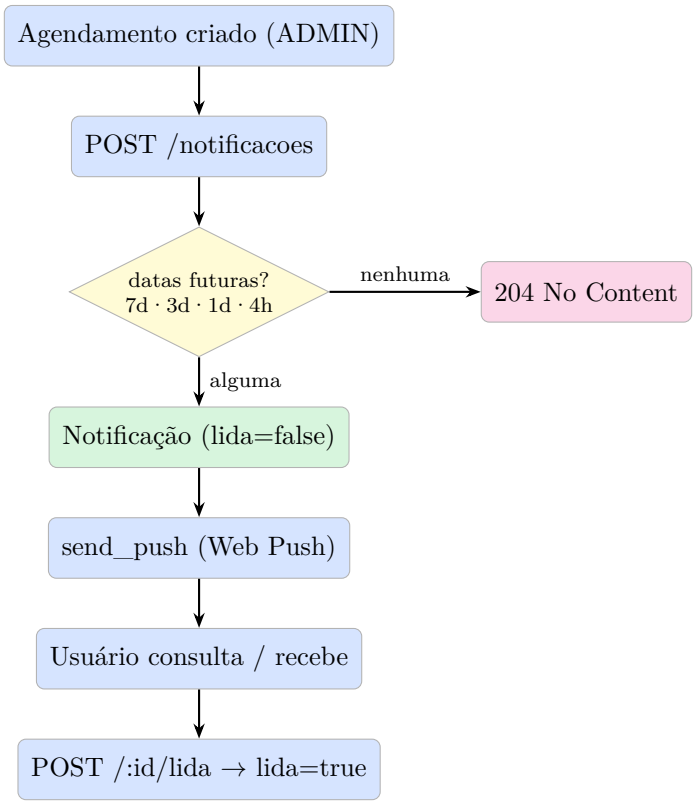
13 Notificações

Cada notificação liga-se a um agendamento (por `id_agendamento`) e a um usuário, com estado `lida`.

13.1 Geração automática

`POST /notificacoes` gera até 4 lembretes: **1 semana**, **3 dias**, **1 dia** e **4 horas** antes. Apenas datas **futuras** são persistidas; se nenhuma for futura, retorna `204`. A criação em lote aplica a mesma regra.

13.2 Ciclo de vida



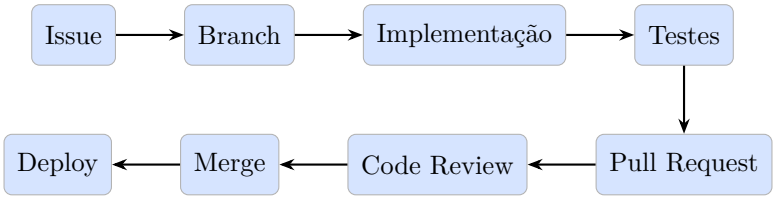
13.3 Web Push e cron

Inscrição: o front assina o push com a chave VAPID pública e envia a `POST /salvar-inscricao` (`PushSubscription`). *Envio*: `send_push(...)` usa `pywebpush` com `VAPID_PRIVATE_KEY`. *Cron*: `manage.py send_reminders` conforme `REMINDER_CRON_SCHEDULE`; `Agendamento.lembrete_enviado` controla o disparo único.

[!] **Atenção. Bug conhecido:** `notification_create` cria `Notificacao(..., user=user)`, mas o campo do model é `usuario` (corrigir para `usuario=user`). Em `notification_create_batch`, as notificações não associam `usuario`, impedindo o push. Verifique também se o comando `send_reminders` existe em `management/commands/`.

14 Desenvolvimento e Manutenção

14.1 Fluxo de contribuição



Padrões: PEP 8 + docstrings (backend); ESLint + Prettier (frontend). Commits descritivos (`feat:`/`fix:`/`docs:`).

14.2 Submódulos

Ao alterar `front`/`back`, o `agendinha` aponta para commits específicos. Após atualizar:

```
cd agendinha
git submodule update --remote front # ou back
git add front && git commit -m "chore: atualiza submodulo front"
```

14.3 Dívida técnica (encontrada no código)

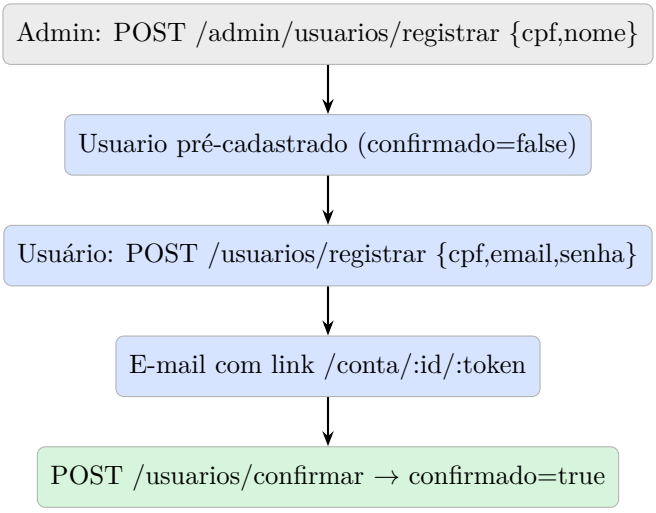
Alta — quebram funcionalidade: (1) notificação usa `user=` em vez de `usuario=`; (2) testes importam `Paciente` inexistente; (3) base URL do front sem o prefixo `/agendinha/api/`. **Alta — segurança:** (4) `SECRET_KEY="agendinha"` fixa; (5) segredos em `runtimeConfig.public`; (6) `.env` versionado. **Média — consistência:** (7) códigos HTTP (500 onde seria 404; 200 onde seria 201); (8) `user_search_by_name` definida duas vezes em `views/user.py`; (9) nomes legados no Docker; (10) dependências não usadas no front; (11) confirmar o comando `send_reminders`.

14.4 Regra de manutenção da documentação

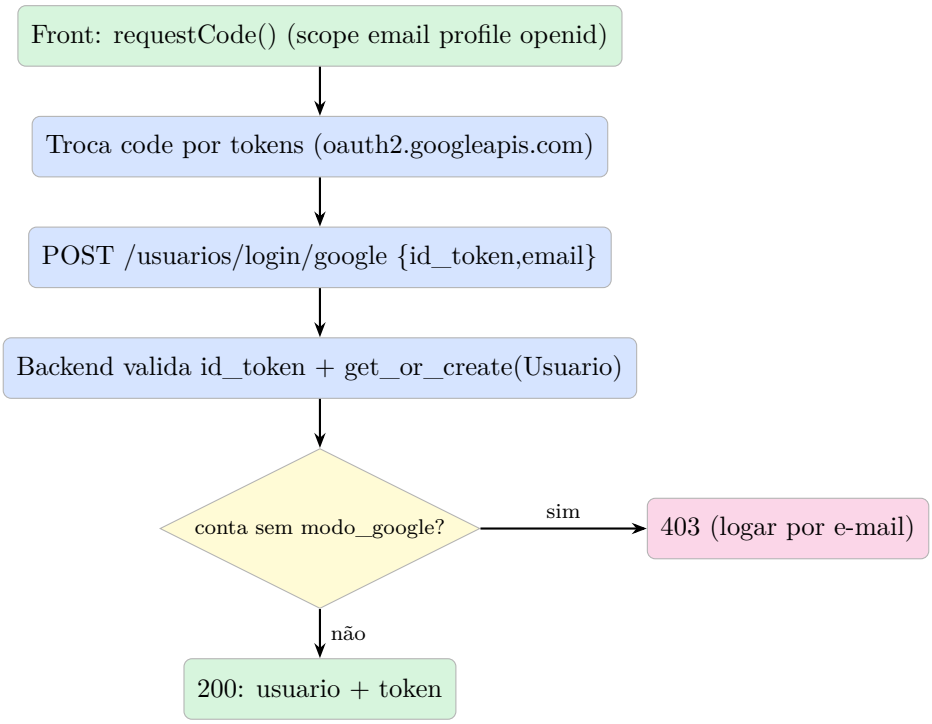
Sempre que uma alteração modificar **rota, entidade, variável de ambiente, regra de negócio ou fluxo relevante**, atualize a seção correspondente desta Wiki no mesmo Pull Request.

15 Fluxogramas complementares

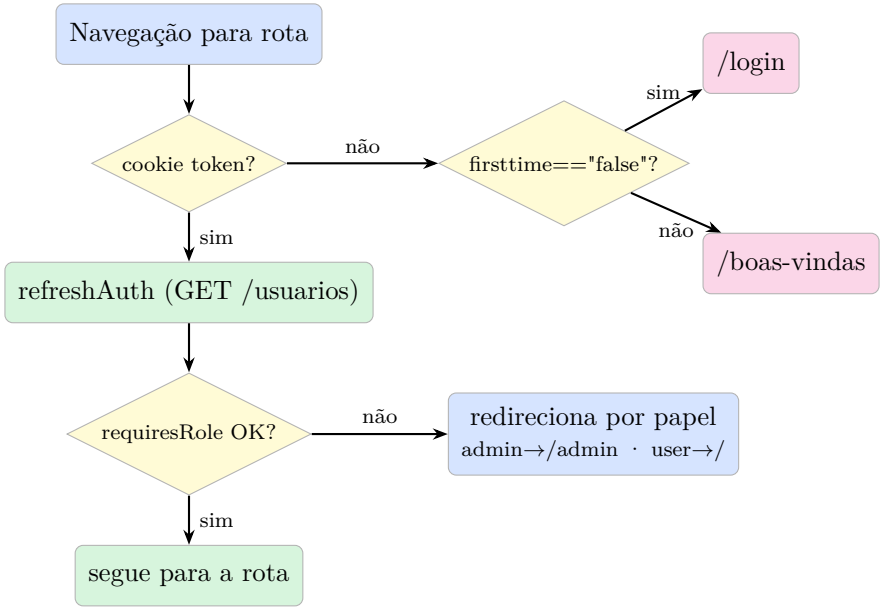
15.1 Cadastro e confirmação de conta



15.2 Login com Google OAuth2



15.3 Proteção de rota no frontend



16 Licenciamento de Software

Esta seção registra as licenças identificadas no levantamento das dependências utilizadas pelo Agendinha. As informações refletem os metadados dos pacotes listados no documento de levantamento e devem ser confirmadas nas fontes oficiais antes de uma distribuição. Este resumo é informativo e não substitui os arquivos legais de licença dos projetos ou das dependências.

16.1 Licença dos próprios projetos

No levantamento realizado, os repositórios `agendinha-web` e `agendinha-backend-simple` não possuem arquivo `LICENSE`. O frontend também não declara um campo `license` no `package.json`; no backend, o README informa apenas que o projeto é mantido pela UnifespCodeLab, o que não constitui uma licença de software.

- **Situação:** licença dos projetos ainda não definida/documentada.
- **Ação pendente:** decidir com a equipe responsável se os projetos serão de código aberto (por exemplo, MIT, Apache-2.0 ou GPL-3.0) ou proprietários; depois, adicionar o arquivo `LICENSE` aos dois repositórios.

A ausência de uma licença explícita não concede automaticamente ao público permissão para usar, modificar ou redistribuir o código. A decisão deve ser registrada antes de divulgar o projeto sob uma licença específica.

16.2 Backend — dependências Python

A tabela apresenta as dependências e versões registradas no levantamento de `requirements.txt`.

Pacote	Versão	Licença identificada
`Django`	5.2.8	BSD-3-Clause
`djangorestframework`	3.15.2	BSD-3-Clause
`psycopg2-binary`	2.9.10	LGPL (confirmar versão/termos aplicáveis)
`djangorestframework-simplejwt`	5.3.1	MIT
`PyJWT`	2.10.1	MIT
`python-dotenv`	1.2.1	BSD-3-Clause
`drf-spectacular`	0.27.2	BSD-3-Clause
`django-cors-headers`	4.5.0	MIT
`bcrypt`	4.2.1	Apache-2.0
`pytest-django`	4.9.0	BSD-3-Clause
`requests`	2.32.3	Apache-2.0
`pillow`	12.1.0	MIT-CMU (HPND)
`django-allauth`	65.14.3	MIT
`cryptography`	46.0.5	Apache-2.0 ou BSD-3-Clause (dupla)
`google-auth`	2.48.0	Apache-2.0
`google-api-python-client`	2.191.0	Apache-2.0
`google-auth-oauthlib`	1.3.0	Apache-2.0
`pywebpush`	2.3.0	MPL-2.0

16.3 Frontend — dependências JavaScript/TypeScript

As faixas de versão abaixo foram transcritas do levantamento de `package.json`. A licença de cada pacote deve ser conferida na versão resolvida pelo arquivo de lock antes de uma distribuição.

16.3.1 Dependências de runtime

Pacote	Faixa/versão	Licença identificada
`@mdi/font`	`^7.4.47`	Apache-2.0 / MIT (ícones e fonte)
`@nuxt/fonts`	`^0.11.4`	MIT
`@pinia/nuxt`	`^0.7.0`	MIT
`@popperjs/core`	`^2.11.8`	MIT
`@schedule-x/theme-default`	`^2.4.1`	Verificar termos do free tier
`@schedule-x/vue`	`^2.3.1`	Verificar termos do free tier
`@vite-pwa/nuxt`	`^1.1.1`	MIT
`eslint-config-prettier`	`^10.1.5`	MIT
`eslint-plugin-prettier`	`^5.4.1`	MIT
`google-auth-library`	`^9.15.1`	Apache-2.0
`moment`	`^2.30.1`	MIT
`pinia`	`2.2`	MIT
`prettier`	`^3.5.3`	MIT
`v-calendar`	`^3.1.2`	MIT
`vue`	`latest`	MIT
`vue-toast-notification`	`^3.1.3`	MIT
`vue3-google-login`	`^2.0.33`	MIT

16.3.2 Dependências de desenvolvimento

Pacote	Faixa/versão	Licença identificada
`@nuxt/eslint`	`^1.4.1`	MIT
`@nuxt/icon`	`^1.14.0`	MIT
`@nuxt/test-utils`	`^3.19.2`	MIT
`@nuxtjs/eslint-module`	`^4.1.0`	MIT
`@types/jest`	`^30.0.0`	MIT
`@vue/test-utils`	`^2.4.6`	MIT
`eslint`	`^9.28.0`	MIT
`happy-dom`	`^20.0.8`	MIT
`jest`	`^30.1.3`	MIT
`nuxt`	`^4.5.2`	MIT
`playwright-core`	`^1.56.1`	Apache-2.0
`ts-jest`	`^29.4.4`	MIT
`typescript`	`^5.8.3`	Apache-2.0
`vite-plugin-vuetify`	`^2.1.1`	MIT
`vitest`	`^3.2.4`	MIT
`vuetify`	`^3.7.0`	MIT

16.4 Pontos de atenção e obrigações

- **Schedule-X:** confirmar os termos exatos de `@schedule-x/vue` e `@schedule-x/theme-default`. O levantamento indica que os conectores Vue/Angular estão no free tier, cujos termos podem diferir de uma licença MIT. Não assumir compatibilidade com MIT sem essa confirmação.

- **psycopg2-binary:** o levantamento identifica LGPL. Registrar essa licença e confirmar o texto aplicável à versão efetivamente utilizada.
- **pywebpush:** utiliza MPL-2.0, uma licença copyleft com obrigações específicas para arquivos cobertos por ela.
- **cryptography:** o levantamento registra dupla licença Apache-2.0 ou BSD-3-Clause; verificar os avisos distribuídos com a versão usada.
- **pillow:** o levantamento identifica MIT-CMU/HPND, descrita como permissiva.
- **Atribuição:** licenças como MIT, BSD e Apache-2.0 podem exigir a preservação de avisos de copyright e textos de licença na redistribuição. A Apache-2.0 também prevê requisitos relacionados a alterações e avisos.

16.5 Arquivos e atualização do inventário

A Wiki é um resumo informativo. Recomenda-se manter, em cada repositório, o arquivo `LICENSE` correspondente à licença escolhida para o próprio projeto e gerar um arquivo `THIRD-PARTY-NOTICES` com os avisos e textos exigidos pelas licenças das dependências. Esses arquivos ainda precisam ser criados/confirmados pela equipe; sua existência não é afirmada por esta documentação.

Para atualizar o inventário do frontend e incluir dependências transitivas, o levantamento sugere:

```
npm install
npx license-checker --summary
npx license-checker --csv --out licenses-web.csv
npx license-checker --json > licenses-web.json
```

O relatório deve ser atualizado quando as dependências mudarem e validado contra os metadados e textos oficiais das versões efetivamente instaladas.

17 Glossário e Próximos Passos

17.1 Glossário

Termo	Significado
Agendinha	Sistema de agenda de consultas/exames (UnifespCode-Lab).
Usuário (`Usuario`)	Conta do paciente/responsável; papel USER ou ADMIN.
Responsável (`Responsavel`)	Pessoa associada ao usuário (nome + telefone).
Agendamento	Consulta/exame marcado.
Notificação	Lembrete de um agendamento, com estado `lida`.
PushSubscription	Inscrição Web Push do navegador.
JWT	Token assinado em HS256 (claims `idUsuario`, `role`, ...).
BCrypt	Hash de senhas no model `Usuario`.
VAPID	Par de chaves que autentica o servidor no serviço de push.
DRF	Django REST Framework.
PWA	Progressive Web App (frontend instalável, service worker).

17.2 Próximos passos sugeridos

Documentação: procedimento oficial de deploy (produção, build Docker corrigido, HTTPS, CORS, CI/CD); exemplos completos de request/response exportados do schema OpenAPI; checklist de release. **Código:** corrigir o campo `usuario` nas notificações; atualizar os testes para `Responsavel`; alinhar a base URL do front; sanear segredos; padronizar códigos HTTP e remover código duplicado/morto; corrigir nomes no Docker e confirmar o `send_reminders`.

17.3 Regra de manutenção

Sempre que uma alteração de código modificar uma **rota, entidade, variável de ambiente, regra de negócio ou fluxo relevante**, atualize a seção correspondente desta Wiki **no mesmo Pull Request**.

Fim da documentação — Wiki Agendinha v1.0